MOODLE => webmail.

- enroll asc , test or 2nd.

Overflow cacept analysis.

Why adc test not cd.

bc. the conditions are totally different.

Overflow => a situation which shows that the result of the ALU didn't fit the reserved space, or that result does not belong to the admissible representation or it is a mathematical nonsense.

CF => overflow for unsigned
OF => overflow for signed

=> mathematically incorrect.

Addition

RSA (regula dreptului la adunare)

(+) 2 neg. nr. ⇒ positive ⇒ nonsense ⇒ OF

(+) 2 pos. nr ⇒ negative ⇒ nonsense ⇒ OF

(-) pos - neg ⇒ neg ⇒ nonsense ⇒ OF = 1

(-) neg - pos ⇒ pos ⇒ — " —

m pozit.
n pozit

m+n pozitions.

AL × AL = AX
⇒ no overflow.

b * b ⇒ b ⇒ CF = 0, OF = 0 „no overflow"

b * b ⇒ w ⇒ CF = 1, OF = 1 „overflow"

Exam:

Which of the 10 operation below will set CF and OF to different values?

- multiplication => NONE

Division. (same independantly of interpretation signed / unsigned)

Which are caditions at division where OF and CF = ?

FATAL ERROR. => CRASH

if 20.000 / 2 > 10.000 not on byte
W b

Divide by 0

0 divide

division overflow

n / 0 → ∞ => division by 0 is overflow.

20.000 / 2 => divide by 0.

Why imul, idiv, but no isub, iadd.

They go the same independantly of the interpretation.

push => ESP = ESP-4

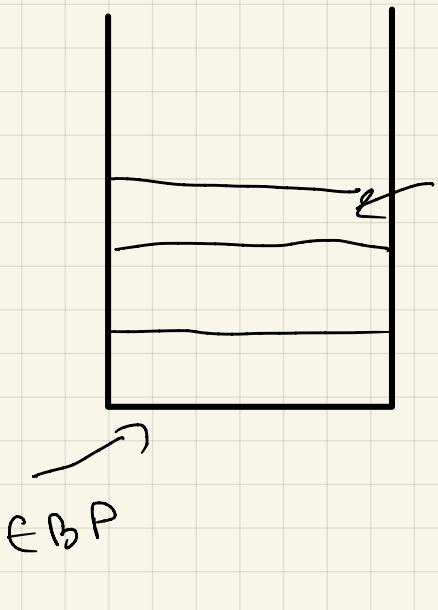
pop => ESP = ESP+4.

xchg d,s. => <d> <=> <D> (1 values)

NO CONSTANTS. / NO BOTH MEMORY ADDRESSES.

push eax <=> sub esp,4
mov [esp], eax

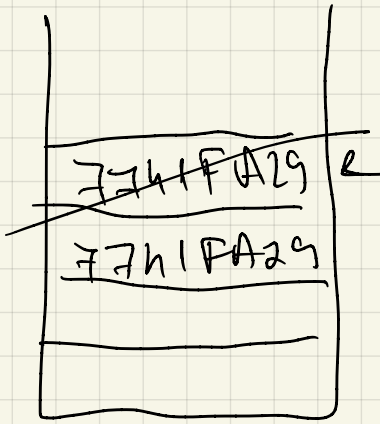
pop eax <=> mov eax, [esp]
add esp,4





ESP = 0019FF7C

pop dword [esp]



ESP = 0019FF4

ESP = 0019FF8

=> top of the stack replaces the top.

239 => '239'

i + '0'

0, 9 => i + '0'

2AB37Fh.

10-15 => i + 'A' - 10

Tablexa db '0123456789ABCDEF'

mov ebx, tablexa ; put offset into ebx.

mov al, 10 ; the pos that you want.

xlat => AL = A.

on SS xlat => tablexa is relative to SS.

register, memory addressed operand ← offset specification formula.

LEA dest, source (Load effective Address)

16 bits TASM.

mov ax, v. ⇔ 32bits NASM

mov ax, offset v

↓

lea ax, v

mov eax, [v]

mov eax, v ; eax ← offset(v)

↓

lea eax, [v]

LEA eax, [ebx + esi * 4 - 7]

~~[~~ → this is exactly what it does but it works.

"mov eax, ebx + esi * 4 - 7" ← syntax error.

How many active DS can we have?

2 (DS + ES), otherwise 1 (for CS and SS)

LAHF

SAHF

PUSHF => pushes flags in the stack

POPF => pops them and puts them in the flags.

```
PUSHF
POP EAX
.....   ← modify the flags.
PUSH EAX
POPF
```

put the back.